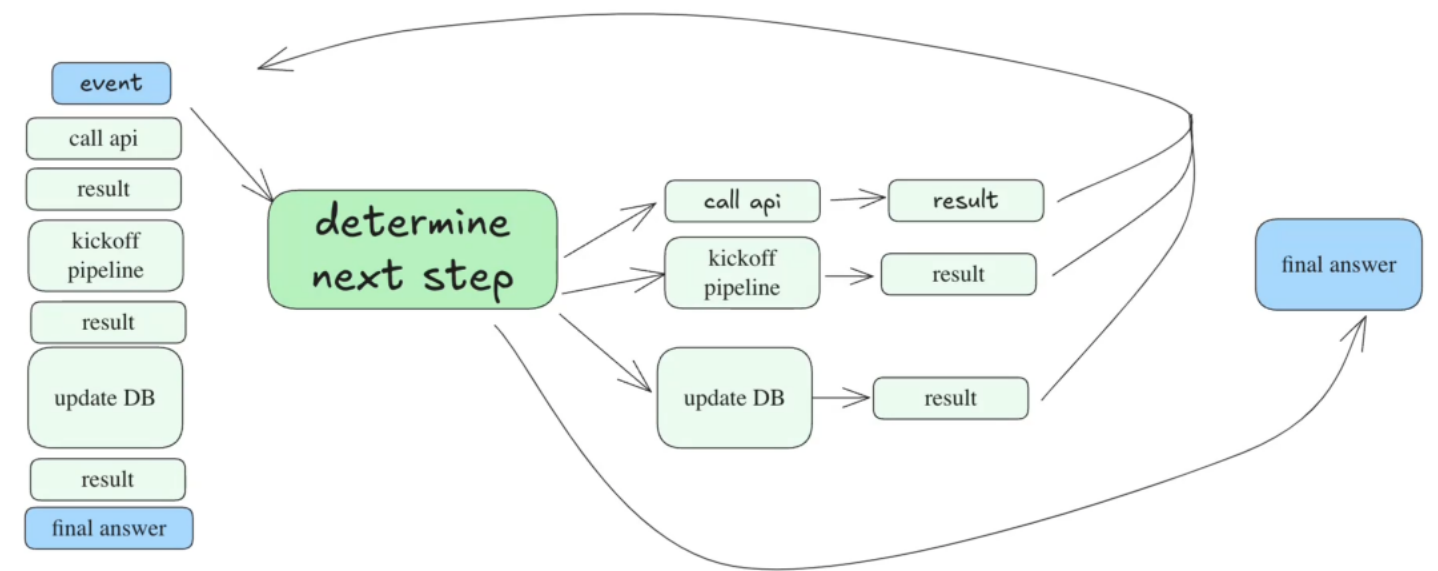


AI Agent智能体架构设计12条关键原则

本文详细解析了AI智能体架构设计的12条关键原则，包括自然语言转工具调用、自主掌控提示词与上下文、工具本质是LLM生成的结构化输出、执行状态与业务状态统一等。

首先我们回顾下 AI 智能体的本质是什么？



如上图所示，AI 智能体的核心在于其如何接收指令、执行任务并做出决策。以下是其关键组成部分：

1. Prompt（提示词）

 Prompt 是指导大语言模型（LLM）如何行动的指令，它定义了 LLM 可以使用的“工具”。Prompt 的输出是一个 JSON 对象，用于描述工作流程中的下一步操作，比如：“工具调用”或“函数调用”。

2. Switch 语句

 Switch 语句根据 LLM 返回的 JSON 内容决定后续操作。这是整个流程中的一个重要环节，用于解析 LLM 的输出并执行相应的动作。

3. 累积的上下文

 累积的上下文用于记录已执行的操作步骤及其运行结果。这一部分是 AI 智能体决策的重要依据，帮助其跟踪任务的进展。

4. For 循环

🔑 For 循环是整个流程的驱动机制。它循环执行以下操作，直至 LLM 返回终止信号（比如：标记为“Terminal”的工具调用或自然语言响应）：将 switch 语句的执行结果加入上下文窗口，并让 LLM 决定下一步动作。

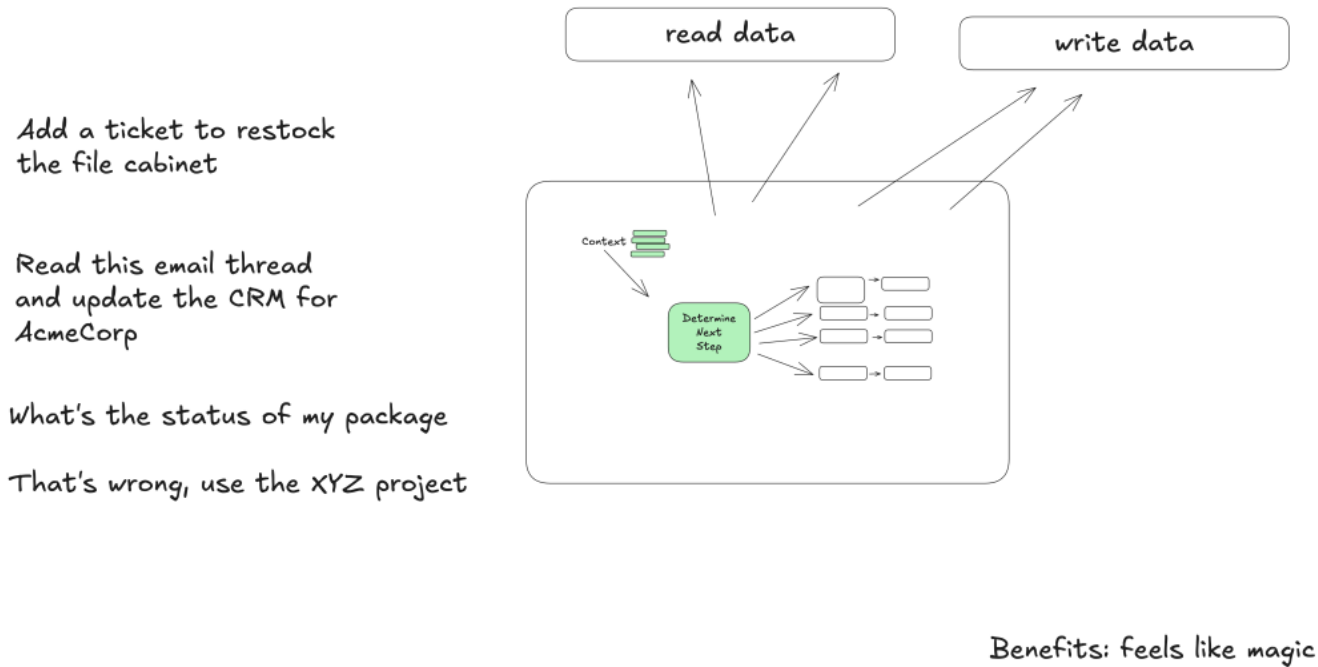
这种设计使得 AI 智能体能够高效地执行任务，同时具备灵活性和适应性。

AI 智能体架构设计的12条原则

1、架构设计原则一：自然语言转工具调用

🔑 将自然语言转换为结构化的工具调用是构建 AI 智能体的常见模式之一。这种模式功能强大，能够使智能体具备推理和执行任务的能力。

Factor 1 - Natural Language -> Tool (API) Calls



这种模式的核心在于实现自然语言与结构化指令之间的精准转换。

2、架构设计原则二：自主掌控提示词

🔑 避免直接使用框架自动生成的提示词。

Factor 2 - Own your prompts

```
Agent:
  role: str
  goal: str
  personality: str
```

```
Task:
  instructions: str
  expected_output: Type[BaseModel]
  tools: list[tool]
```

```
agent.run(task)
```

black box

I don't know what's better, but I know you want to be able to try EVERYTHING

```
Open Playground ↗
function DetermineNextStep()
    // to keep this clean, make the client turn the thread into a prompt-ready string,
    // didn't wanna solve that in jinja (although long term that's probably the best solution)
    thread: string
    -> ClarificationRequest | CreateIssue | ListTeams | ListIssues | ListUsers | DoneForNow | AddComment |
    client CustomGPT4o

    prompt #
    {{ _.role("system") }}

    You are a helpful assistant that helps the user with their linear issue management.
    You work hard for whoever sent the inbound initial email, and want to do your best
    to help them do their job by carrying out tasks against the linear api.

    Before creating an issue, you should ensure you have accurate team/user/project ids.
    You can list_teams and list_users and list_projects functions to get ids.

    If you are BCC'd on a thread, assume that the user is asking you to look up the related issue and

    Always think about what to do first, like:

    - ...
    - ...
    - ...

    {{ _.role("user") }}
```

full control

Benefits: Flexibility, Optimization

一些框架会提供这种“黑箱式”方案：

```
agent = Agent(
    role="...",
    goal="...",
    personality="...",
    tools=[tool1, tool2, tool3]
)
```

```
task = Task(
    instructions="...",
    expected_output=OutputModel
)
```

```
result = agent.run(task)
```

这种方案虽然能让你快速上手并应用顶尖的提示词模板，但后期很难进行精准调整或实施逆向工程，难以确保模型接收到的指令完全符合预期。

相反，你应该像对待核心业务代码一样对待提示词：

```
function DetermineNextStep(thread: string) -> DoneForNow | ListGitTags | DeployBackend | DeployFrontend | RequestMoreInformati:
prompt #"
  {{ _.role("system") }}

  You are a helpful assistant that manages deployments for frontend and backend systems.
  You work diligently to ensure safe and successful deployments by following best practices
  and proper deployment procedures.

  Before deploying any system, you should check:
  - The deployment environment (staging vs production)
  - The correct tag/version to deploy
  - The current system status

  You can use tools like deploy_backend, deploy_frontend, and check_deployment_status
  to manage deployments. For sensitive deployments, use request_approval to get
  human verification.

  Always think about what to do first, like:
  - Check current deployment status
  - Verify the deployment tag exists
  - Request approval if needed
  - Deploy to staging before production
  - Monitor deployment progress

  {{ _.role("user") }}

  {{ thread }}

  What should the next step be?
"#
}
```

3、架构设计原则三：自主掌控上下文窗口

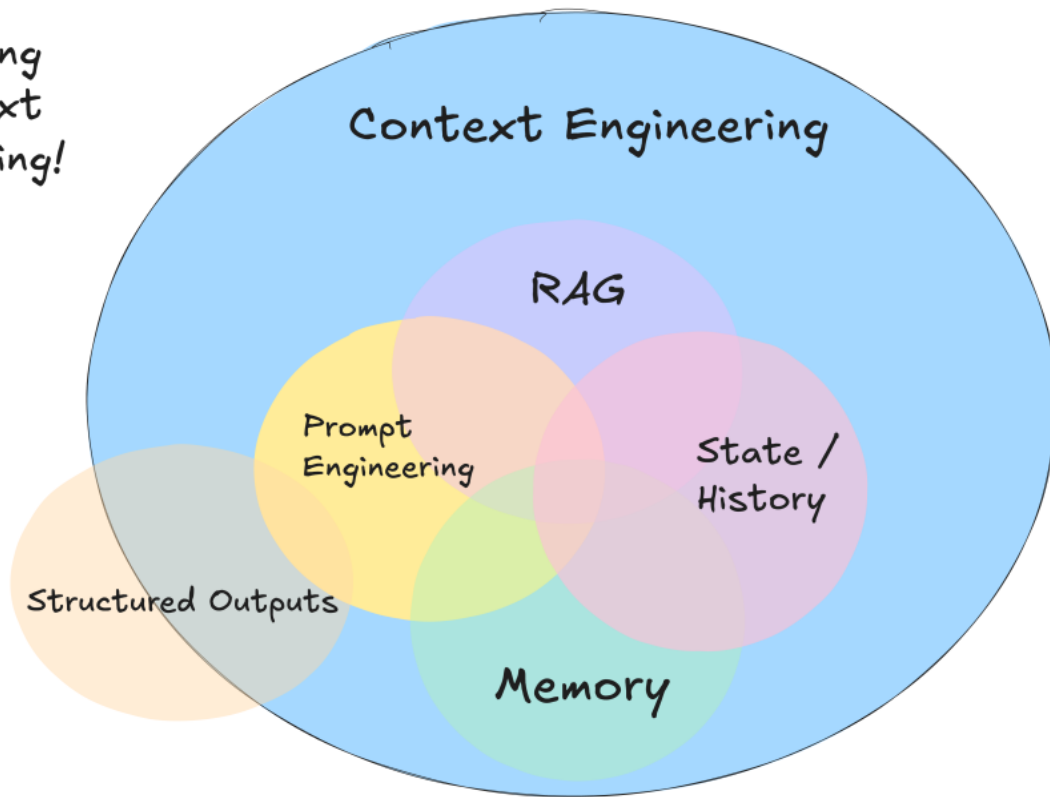
 在与大语言模型交互时，上下文传递不必局限于标准消息格式。对于 AI 智能体而言，每次向 LLM 提供输入的本质是：“这是当前情况，接下来该怎么做？” 一切皆为上下文工程。大语言模型本质上是无状态函数，其作用是将输入转化为输出。要获得优质输出，就必须提供优质的输入。

构建优质上下文需要包含以下要素：

1. **提示词与指令**：为模型提供明确的操作指引。
2. **外部数据源**：检索到的文档或知识（比如：使用 RAG 技术）。
3. **历史状态**：过往的工具调用记录、执行结果等操作痕迹。
4. **记忆系统**：与当前对话或事件相关但独立的历史记录。
5. **输出结构化指令**：规定模型返回数据的格式规范。

通过优化上下文输入，可以显著提升大语言模型的输出质量和任务执行效率。

Everything
is Context
Engineering!



4、架构设计原则四：「工具」本质上只是 LLM 生成的结构化输出



“工具”的设计无需过于复杂。其核心是大语言模型生成的结构化输出，用于触发确定性的代码执行。

Factor 4 - Tools are Structured Output

```
const openai_tools: ChatCompletionTool[] = [
  {
    type: "function",
    function: {
      name: "multiply",
      description: "multiply two numbers",
      parameters: {
        type: "object",
        properties: {
          a: { type: "number" },
          b: { type: "number" },
        },
        required: ["a", "b"],
      },
    },
  },
  {
    type: "function",
    function: {
      name: "add",
      description: "add two numbers",
      parameters: {
        type: "object",
        properties: {
          a: { type: "number" },
          b: { type: "number" },
        },
        required: ["a", "b"],
      },
    },
  },
];
```

JSON Schema

Prompt-Only

```
What should the next step be?

Answer in JSON using any of these schemas:
{
  // you can request more information from me
  intent: "request_more_information",
  message: string,
} or {
  intent: "create_issue",
  issue: {
    title: string,
    description: string,
    team_id: string,
    // name of the team to create the issue in
    team_name: string or null,
    project_id: string or null,
    // name of the project to create the issue in
    project_name: string or null,
    assignee_id: string or null,
    // name of the user to assign the issue to
    assignee_name: string or null,
    labels_ids: string[],
    labels_names: string[],
    // The priority of the issue.
    // 0 = No priority, 1 = Urgent, 2 = High, 3 = Normal, 4 = Low.
    priority: int or null,
  },
} or {
  intent: "list_teams",
  // what is your goal or desired outcome with this operation
  query_intent: string or null,
} or {
```

I don't know what's better, but I know you want to be able to try EVERYTHING

Benefits: Flexibility, Prompt Control

以 `CreateIssue` 和 `SearchIssues` 这两个工具为例，让大语言模型（LLM）“调用工具”，本质上是让它输出一个可解析的 JSON 对象，该对象对应要执行的具体操作。

这种模式非常简洁，包含以下三个步骤：

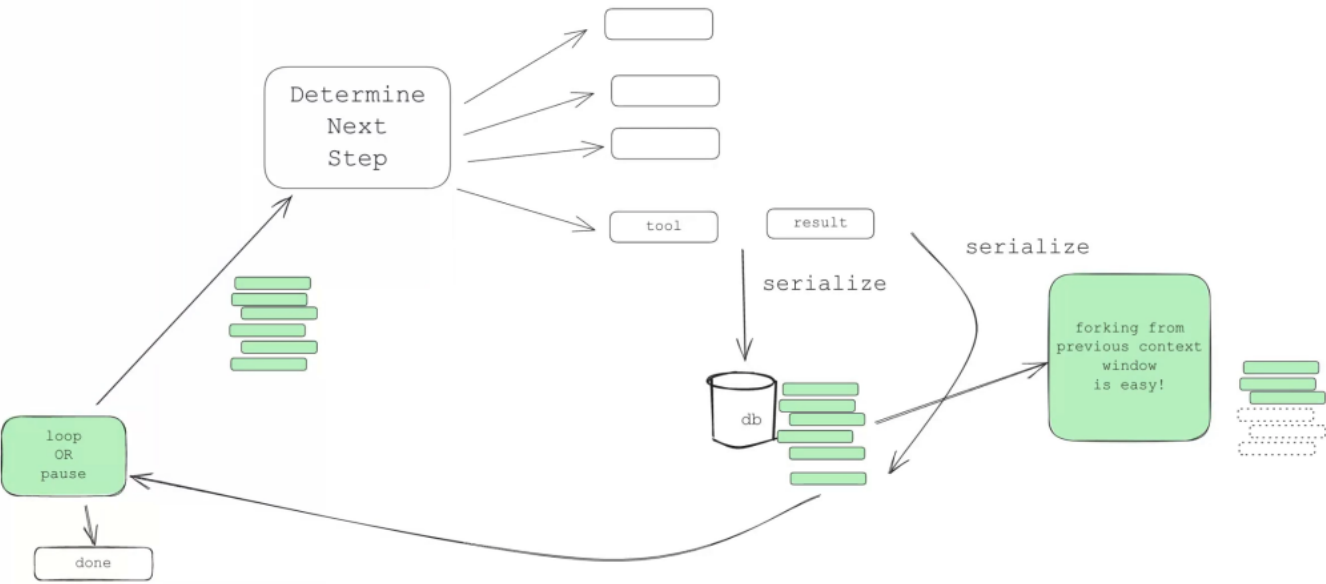
- 1. LLM生成结构化的 JSON 输出。
- 2. 使用确定性代码执行对应操作（比如：调用外部 API）。
- 3. 捕获执行结果并反馈到上下文中。

这种设计实现了 LLM 决策逻辑与应用程序执行逻辑的清晰分离：LLM 负责决定“做什么”，而代码则掌控“怎么做”。即使 LLM 调用了某个工具，具体的执行方式也不必每次都严格对应单一函数。

5、架构设计原则五：执行状态与业务状态的统一

✏️ 即使在 AI 领域之外，许多基础设施系统也在尝试分离“执行状态”与“业务状态”。对于 AI 应用来说，这种分离可能需要复杂的抽象机制来跟踪当前步骤、下一步骤、等待状态、重试次数等信息。虽然这种分离带来的复杂性在某些情况下可能是值得的，但对你的具体使用场景来说，可能是一种过度设计。

Factor 5 - Unify Execution State and Business State



你可以根据自己的需求来决定最适合的方案。不要认为必须将这两者分开管理。

更清晰的定义如下：

- **执行状态（Execution state）**：包括当前步骤、下一步骤、等待状态、重试次数等。
- **业务状态（Business state）**：指智能体工作流中已发生的事件，例如 OpenAI 消息列表、工具调用及结果列表等。

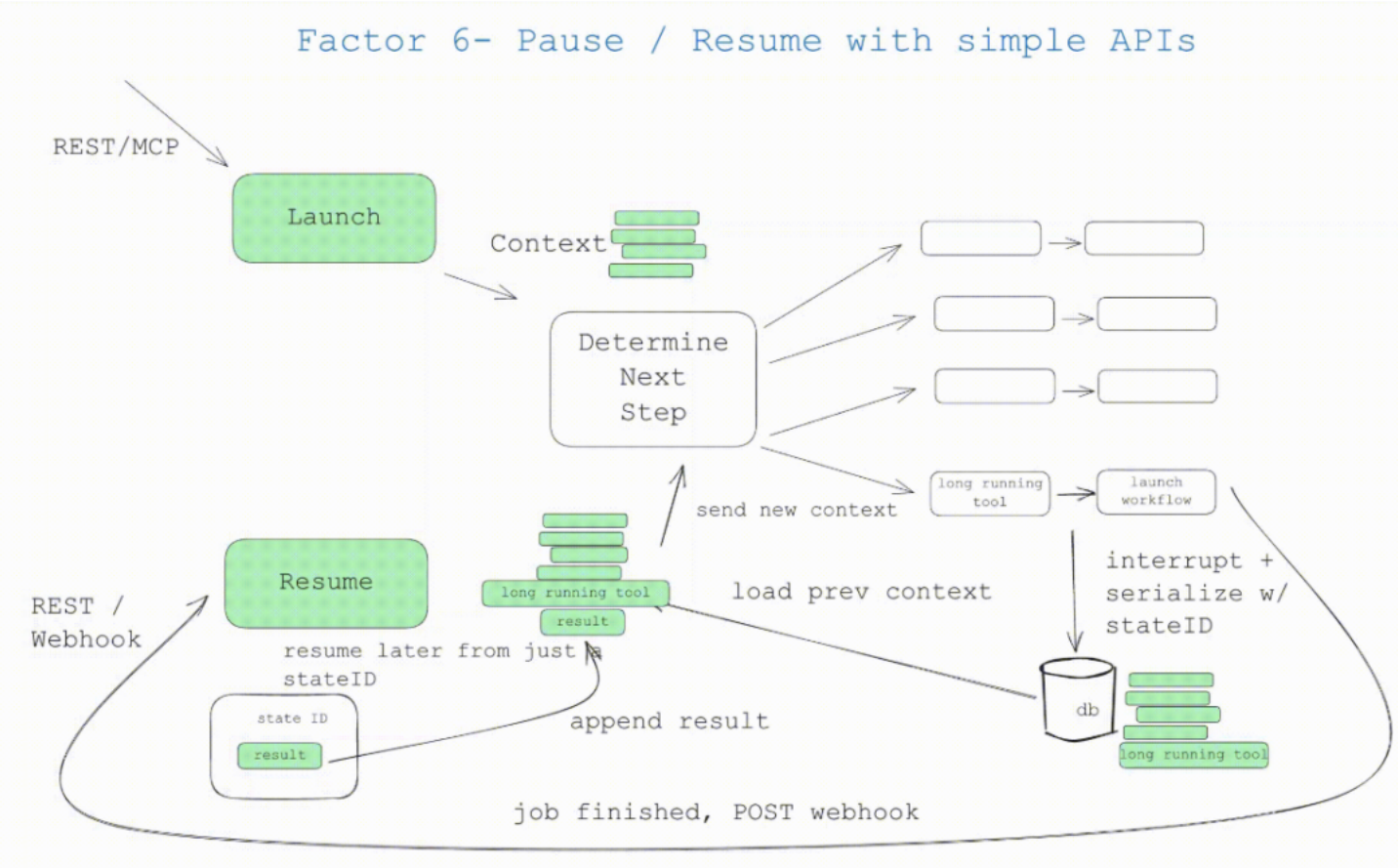
如果可能的话，尽量简化设计——尽可能将执行状态和业务状态统一起来。

实际上，通过精心的工程设计，你可以直接从上下文窗口推断出所有执行状态。在大多数情况下，执行状态（如当前步骤、等待状态等）本质上只是已发生事件的元数据（metadata）。

当然，有些信息（如会话 ID、密码上下文等）可能无法放入上下文窗口，但你的目标应该是尽量减少这类例外。通过遵循架构设计原则3，你可以精准地控制哪些信息真正输入给 LLM。

6、架构设计原则六：通过简单的 API 实现启动/暂停/恢复

 AI 智能体本质上是一种程序，其生命周期管理应符合我们对普通程序的期望：能够方便地启动、查询、恢复和终止。



必须确保终端用户、应用程序、业务流程以及其他 AI 智能体都可以通过轻量级 API 接口快速部署新的智能体实例。此外，AI 智能体及其编排的确定性代码应具备长期运行时的自动暂停能力。

7、架构设计原则七：通过工具调用实现人机协同

 默认情况下，LLM API 在生成模型响应时，首先需要做出一个关键且高风险的选择：是返回纯文本内容，还是返回结构化数据？

Factor 7 - Contact Human with tools

content: "the weather in tokyo is 57 and rainy"

OR

content: "for what postal code?"

OR

tool_calls: [...]

model switches between
tools and plaintext

```
tool_calls: [{  
  function: "final_answer",  
  arguments: {  
    message: "the weather in tokyo is 57 and rainy",  
  }  
}]
```

OR

```
tool_calls: [{  
  function: "request_clarification",  
  arguments: {  
    message: "for what postal code?",  
  }  
}]
```

OR

```
tool_calls: {  
  function: "check_weather_in_city",  
  arguments: {  
    "city": "tokyo",  
    "postal_code": "12345"  
  }  
}
```

everything
is tools

Final answer vs. I need input vs. call tool is fiddly

Benefits: Clear instructions for the model, inner vs. outer loop,
access to multiple humans

系统会将大量决策权重集中在首个词元的选择上。比如：在查询“东京天气”时，首个词元可能是：

- “东京”

而在 `fetch_weather`（获取天气）功能中，则可能是表示 JSON 对象开头的特殊词元：

- |JSON>

更优的方案是让大语言模型始终输出 JSON 结构化数据，然后通过自然语言词元（如 `request_human_input` 或 `done_for_now`）来声明意图，而不是依赖类似 `check_weather_in_city` 这样的“标准”工具。

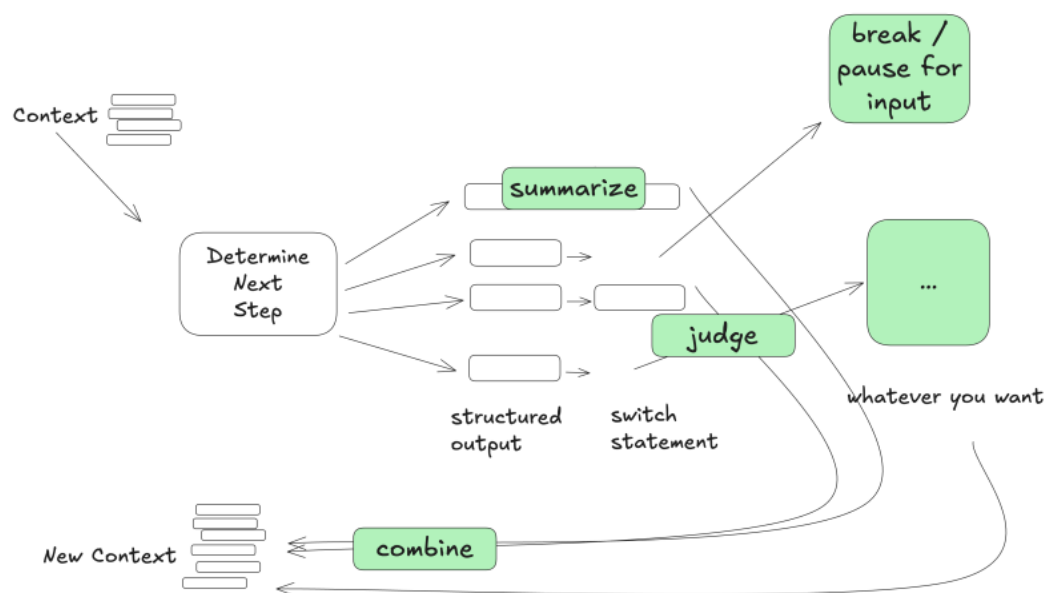
需要注意的是，这种方案可能不会直接提升系统的性能指标，但它必须通过持续实验来验证。工程师应该保留尝试非常规手法的自由度，这是找到最优解的必经之路。

8、架构设计原则八：掌控你的控制流



如果你能够掌控控制流，就能实现更灵活、更强大的功能。

Factor 8 - Own Your Control Flow - BYO Switch Statement



Benefits: Interrupts, Process Control

根据具体应用场景构建专属的控制逻辑，精准匹配业务需求。当特定类型的工具调用需要中断循环以等待人工响应或执行长时任务（例如训练流水线）时，你可以设计灵活的中断机制。建议集成以下自定义功能：

- **工具调用结果的摘要与缓存：**快速检索和复用结果，避免重复计算。
- **使用 LLM 对结构化输出进行校验和评估：**确保输出的准确性和可靠性。
- **上下文窗口压缩或其他内存管理方案：**优化内存使用，提升系统效率。
- **全链路日志追踪与性能监控：**实时监控系统运行状态，快速定位问题。
- **客户端速率限制策略：**防止过载，保障系统稳定运行。
- **持久化的休眠/暂停/事件等待机制：**支持长时间任务的灵活暂停与恢复。

通过这些自定义功能，你可以构建出更高效、更可靠的智能体系统，满足复杂多变的业务需求。

9、架构设计原则九：将错误信息压缩至上下文窗口

📌 这一点虽然简短，但值得一提。智能体的优势之一在于“自我修复”--对于短任务，大语言模型（LLM）可能会调用某个失败的工具。优秀的 LLM 通常能够读取错误信息或 Stack Trace 信息，并在后续工具调用中调整策略。

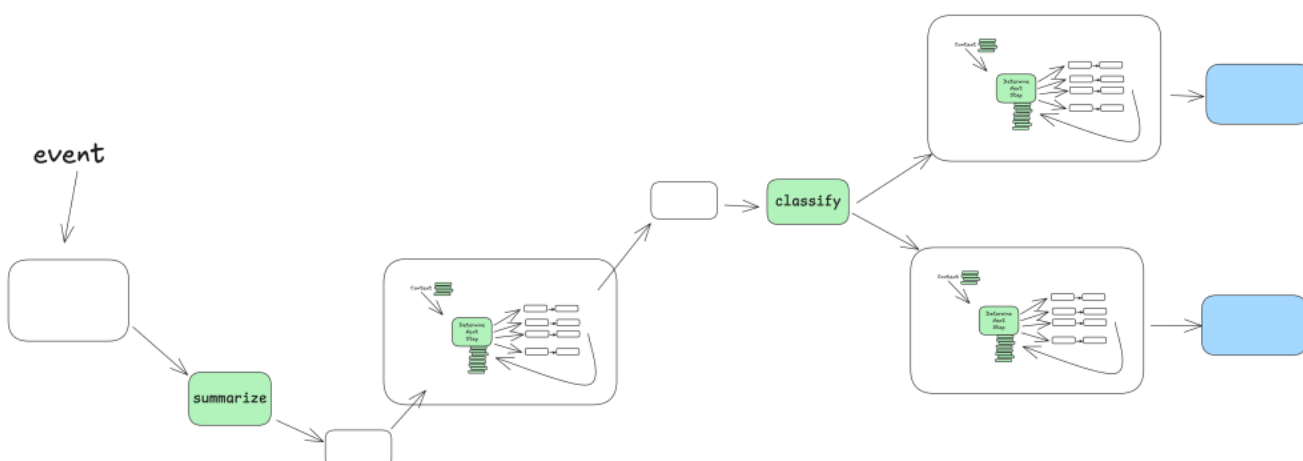
大多数框架已实现这一功能，但你也可以只做到这一点。

```
1 thread = {"events": [initial_message]}
2
3 while True:
4     next_step = await determine_next_step(thread_to_prompt(thread))
5     thread["events"].append({
6         "type": next_step.intent,
7         "data": next_step,
8     })
9     try:
10         result = await handle_next_step(thread, next_step) # our switch statement
11     except Exception as e:
12         # if we get an error, we can add it to the context window and try again
13         thread["events"].append({
14             "type": 'error',
15             "data": format_error(e),
16         })
17         # loop, or do whatever else here to try to recover
18
```

10、架构设计原则十：小型化、功能聚焦的 AI 智能体

与其构建一个试图包办一切的大型 AI 智能体，不如开发专注于某一单一功能的小型 AI 智能体。AI 智能体只是更大系统中的一个基础组件，而这个系统主要由确定性因素驱动。

Factor 10 - Small, Focused Agents



10-100 tools, 3-10 steps
as part of larger pipeline

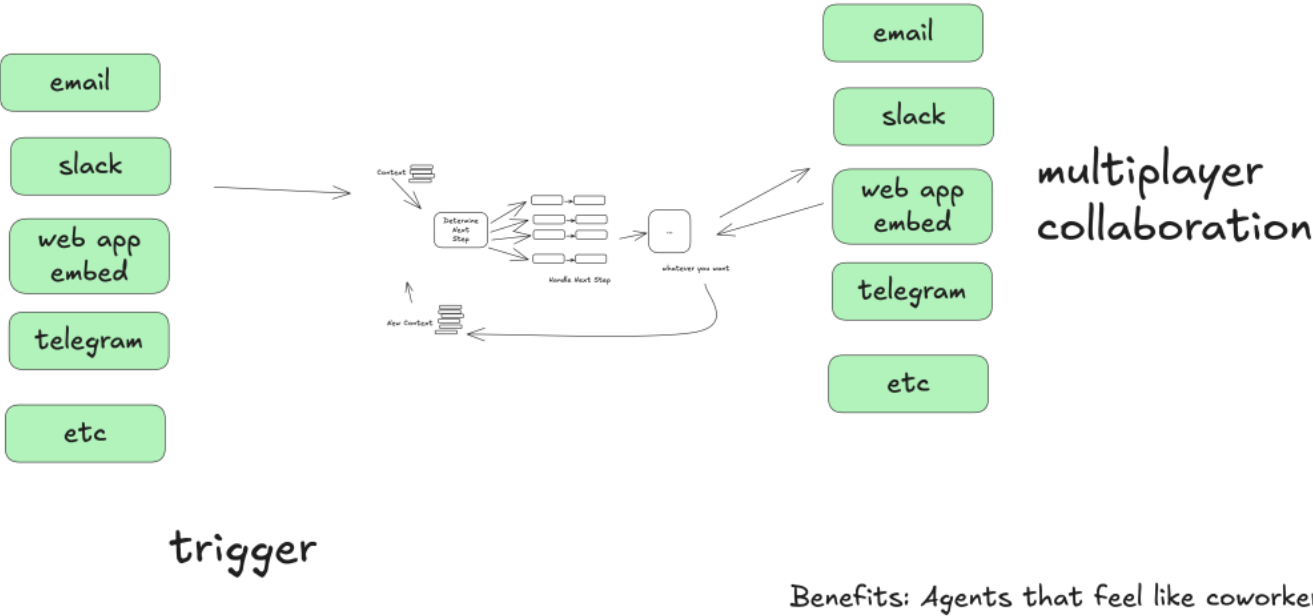
Benefits: better performance

关键在于大语言模型（LLM）的局限性：任务越庞大复杂，所需步骤就越多，上下文窗口也就越长。随着上下文的增长，大语言模型更容易迷失方向或偏离重点。通过将 AI 智能体的功能限定在特定领域（3-10个步骤，最多20个任务步骤），我们可以保持上下文窗口的可管理性，从而确保大语言模型的高效运行。

11、架构设计原则十一：AI 智能体系统支持多入口触发，且响应与触发渠道保持一致

✍ 在落实了原则 6 和原则 7 之后，你可以无缝集成这一原则的方案。

Factor 11 - Trigger From Anywhere, Meet Users where they are



Benefits: Agents that feel like coworkers

支持用户通过 Slack、邮件、短信等任意渠道唤醒智能体，并确保响应始终在原对话流中完成。

优势：

- **满足用户需求：**打造拟人化的 AI 应用体验，让用户感受到如同真人协作般的自然交互，至少达到数字同事的交互水平。
- **外循环 AI 智能体：**支持非人工触发（比如：系统事件、定时任务、故障告警等），AI 智能体可自主运行 5-90 分钟，并在关键决策节点自动请求人工介入（如需审批、获取反馈或请求协助）。
- **高风险操作授权机制：**通过快速人工复核机制，授权 AI 智能体执行高风险操作（比如：外发邮件、生产数据更新等）。明确且标准化的管控体系既能保障操作的可追溯性，又能提升 AI 智能体执行复杂任务的可靠性。

12、架构设计原则十二：将 AI 智能体设计为符合“无状态化归并器”模式



dex ✓
@dexhorthy



but actually, if we kind of accept that "an AI Agent is basically a for loop", then what's the lisp version of this?

> an AI Agent is a foldL where the accumulator is a context window, and the reducer is an LLM DetermineNextStep + a switch statement on how to handle it



dex ✓ @dexhorthy · Jan 5

say it with me now...a for loop is not a framework

总之，在 AI 智能体应用企业级落地过程中，使用这12 条极简原则教会开发者如何把 LLM 从“聊天玩具”升级成可上线、可运维、可扩展的生产级 AI 智能体。

1. 自然语言→结构化工具调用：让 LLM 输出 JSON 去驱动真实 API。
2. 提示词即代码：自己写、版本化、可回滚，拒绝黑盒模板。
3. 上下文即一切：把提示、工具结果、记忆、外部数据全部塞进可控的上下文窗口。
4. 工具 = LLM 的 JSON：LLM 只负责“下命令”，真正干活的是后端确定性代码。
5. 执行状态 ≈ 业务状态：别把流程元数据和业务数据拆得太细，能省则省。
6. 生命周期可暂停：提供简单 API 支持启动、挂起、恢复、终止。
7. 人机协同靠工具：用 `request_human_input` 这类工具把“人”当作可调用的资源。
8. 控制流随你改：允许在循环里随时插钩子，做校验、限流、日志、压缩等自定义逻辑。
9. 错误信息进上下文：让 LLM 看到报错就能自我修正。
10. 小步快跑：3-10 步完成一个任务，宁可拆多个小 AI 智能体也别写“万能大管家”。
11. 多渠道触发，原路返回：Slack、邮件、Webhook 都能唤醒 AI 智能体，回复永远回到同一通道。
12. 无状态归并器：AI 智能体本身无状态，状态只存在上下文或外部存储，可随时横向扩展。